

statnnet: Regression-Style Statistical Summaries for Neural Networks Fitted with nnet

by Andrew McInerney and Kevin Burke

Abstract The `nnet` package provides a stable and widely used implementation of single-hidden-layer feedforward neural networks in R, but its output is primarily concerned with fitting and prediction. The `statnnet` package extends fitted `nnet` objects with regression-style statistical summaries for small, parsimonious networks. It provides Wald-type summaries for individual weights and groups of input weights, partial covariate-effect plots with uncertainty estimates, and diagnostic information concerning optimisation and covariance estimation. The scope is deliberately restricted to supported models fitted using `nnet`; the package is not intended to provide general-purpose inference for deep or otherwise highly parameterised neural networks. This paper describes the statistical quantities computed by `statnnet`, explains the package design and its limitations, and demonstrates the workflow using an applied regression example.

1 Introduction

Feedforward neural networks (FNNs) are flexible non-linear models that are widely used for prediction and function approximation (LeCun et al., 2015). However, from a statistical perspective, they can also be viewed as nonlinear regression models, in which a vector of inputs is mapped to a conditional mean of a response through a sequence of weighted linear combinations and non-linear activation functions. In R, the `nnet` package (Venables and Ripley, 2002) provides a long-established implementation for fitting single-hidden-layer FNNs, with optional skip-layer connections and weight decay. An object returned by `nnet()` contains the fitted weights, fitted values, residuals, objective value, and convergence code, and can also contain a Hessian matrix. However, the standard output is primarily concerned with fitting and prediction and does not provide covariate-level standard errors, Wald-type summaries, analysis-of-variance-style tables, or covariate-effect summaries.

This creates a practical obstacle for statisticians who wish to use shallow FNNs as statistical modelling tools rather than only as predictive algorithms. In a classical regression workflow, a fitted model can be inspected using familiar functions such as `summary()`, `anova()`, `predict()`, and `plot()`. These functions help answer two questions that naturally arise when modelling the relationship between a response and a set of covariates: whether a covariate appears to contribute to the fitted model, and how the fitted response changes with that covariate. For a fitted FNN, answering these questions typically requires the user to extract weights manually, reconstruct the model matrix, compute approximate covariance matrices, write custom code for grouped summaries, or use model-agnostic interpretability tools. Model-agnostic approaches, such as Shapley values (Strumbelj and Kononenko, 2010) and locally interpretable model explanations (LIME) (Ribeiro et al., 2016), are useful for explaining fitted predictions across a wide range of model classes. However, they treat the fitted network as a black box and do not exploit the particular parameterisation of a shallow neural network as a non-linear regression model. The aim of `statnnet` is to add these model-based, regression-style summaries to a precisely defined subset of models fitted using `nnet`. Restricting the package to one fitting engine avoids treating materially different neural-network implementations as interchangeable and allows the parameter ordering, activation functions, objective, convergence information, and Hessian calculations used by `nnet` to be handled explicitly.

The statistical methodology underlying `statnnet` was developed in McInerney and Burke (2023), which treats suitably parsimonious shallow FNNs as statistical models, derives Wald-type tests for individual weights and covariate-level groups of weights, and introduces partial covariate-effect (PCE) plots as neural-network analogues of regression coefficients. The aim of the present paper is to describe the corresponding R implementation and to show how these summaries can be obtained through a workflow that will be familiar to users of standard R modelling functions. The `statnnet` package is a statistical-summary extension to `nnet`: it does not introduce a separate neural-network optimiser or attempt to provide a common representation for several fitting frameworks. Instead, it takes a supported fitted "nnet" object and constructs a "statnnet" object containing the additional information required for covariance estimation, grouped Wald-type summaries, PCE plots, and diagnostic checks. The resulting object can be inspected using S3 methods for printing, summaries, analysis-of-variance-style tables, plotting, and prediction.

By augmenting the established "nnet" workflow with a consistent set of S3 methods, `statnnet`

makes supported shallow FNNs behave more like standard statistical model objects in R. The paper is organised as follows. Section 2.2 introduces the FNN parameterisation and the statistical quantities used by **statnnet**. Section 2.3 positions the package relative to existing neural-network fitting and model-interpretability packages. Section 2.4 demonstrates the package workflow using an applied regression example. Section 2.5 summarises the supported model objects and S3 methods. Section 2.6 discusses practical limitations and appropriate interpretation of the resulting summaries. Section 2.7 concludes.

2 Statistical background

This section summarises the statistical quantities computed by **statnnet**. A fuller development of the methodology, including the derivation of the Wald-type summaries, simulation evidence, and practical recommendations, is given in [McInerney and Burke \(2023\)](#). Here, we introduce only the notation and definitions required to understand the package output.

The package is designed for supported single-hidden-layer models fitted using **nnet** that can be viewed as non-linear regression models. Let y_i denote the response for observation $i = 1, \dots, n$, and let $x_i = (x_{i0}, x_{i1}, \dots, x_{ip})^T$ denote the corresponding vector of covariates, where $x_{i0} \equiv 1$ is an intercept term. For a single-hidden-layer FNN with q hidden nodes, the conditional mean can be written as

$$\mathbb{E}(y_i | x_i) = \text{NN}(x_i, \theta),$$

where

$$\text{NN}(x_i, \theta) = \phi_o \left[\gamma_0 + \sum_{k=1}^q \gamma_k \phi_h \left(\sum_{j=0}^p \omega_{jk} x_{ij} \right) \right].$$

Here, ω_{jk} is the weight connecting input j to hidden node k , γ_k is the weight connecting hidden node k to the output node, γ_0 is the output intercept, and θ denotes the full vector of model parameters. The functions $\phi_h(\cdot)$ and $\phi_o(\cdot)$ are the hidden-layer and output-layer activation functions, respectively. For continuous responses, $\phi_o(\cdot)$ is typically the identity function; for binary responses, it is typically the logistic function.

The package works with fitted FNNs and computes model-based summaries from the fitted parameter vector, the model matrix, the response, and the assumed response distribution. For continuous responses, **statnnet** uses a Gaussian likelihood; for binary responses, it uses a Bernoulli likelihood. When a ridge penalty is used during fitting, the penalised log-likelihood can be written as

$$\ell_\lambda(\theta) = \sum_{i=1}^n \log f(y_i | x_i, \theta) - \lambda \|\tilde{\theta}\|_2^2,$$

where $f(y_i | x_i, \theta)$ is the assumed density or probability mass function, λ is the ridge penalty, and $\tilde{\theta}$ denotes the vector of weights to which weight decay is applied in the fitted "nnet" objective. This corresponds to the weight-decay penalty commonly used when fitting neural networks. The fitted parameter vector is then combined with an estimated covariance matrix for $\tilde{\theta}$ to construct the Wald-type summaries and PCE plots described below. In **statnnet**, this covariance matrix is obtained from the local curvature of the objective at the fitted parameter vector, using the Hessian-based approximation described in [McInerney and Burke \(2023\)](#). The resulting covariance and Wald-type quantities are conditional on the fitted architecture, penalty, and local solution. They are not post-selection summaries and do not account for uncertainty arising from choosing the architecture, penalty, or initialisation.

Wald-type summaries

One aim of **statnnet** is to provide regression-style summaries for supported "nnet" fits. Given an estimate $\hat{\theta}$ and an estimated covariance matrix $\text{Cov}(\hat{\theta})$, the package computes Wald-type summaries for both individual weights and groups of weights.

For an individual parameter θ_m , the null hypothesis

$$H_0 : \theta_m = 0$$

is summarised using the statistic

$$W_m = \frac{\hat{\theta}_m^2}{\widehat{\text{Cov}}(\hat{\theta})_{mm}},$$

which is compared to a χ_1^2 reference distribution. This produces a parameter-level summary similar in form to the output from classical regression models. However, individual neural-network weights are usually difficult to interpret in isolation, because the effect of an input variable is distributed across multiple connections and depends on the rest of the fitted network.

For this reason, **statnnet** also provides covariate-level Wald-type summaries. For a single-hidden-layer FNN, the contribution of covariate x_j to the hidden layer is represented by the vector

$$\omega_j = (\omega_{j1}, \dots, \omega_{jq})^T,$$

which contains all weights connecting covariate j to the hidden nodes. The package therefore computes a multiple-parameter Wald-type statistic for

$$H_0 : \omega_j = 0_q.$$

If S_j is the matrix selecting the elements of θ corresponding to ω_j , then the relevant covariance matrix is

$$\widehat{\Sigma}_{\omega_j} = S_j \widehat{\text{Cov}}(\hat{\theta}) S_j^T.$$

The corresponding statistic is

$$W_j = \hat{\omega}_j^T \widehat{\Sigma}_{\omega_j}^{-1} \hat{\omega}_j.$$

This statistic is compared to a chi-squared reference distribution with the effective degrees of freedom described in [McInerney and Burke \(2023\)](#); this is approximately q when the penalty is small. It provides an input-level summary that is closer in spirit to testing whether a covariate contributes to the fitted model.

For categorical covariates represented by several dummy variables, **statnnet** extends this idea by testing all associated input nodes jointly. For example, if a categorical covariate with $l + 1$ levels is represented using l dummy variables, and the network has q hidden nodes, the corresponding grouped test involves lq input-to-hidden weights. The corresponding effective degrees of freedom are approximately lq when the penalty is small. This gives an analysis-of-variance-style summary for categorical predictors, allowing the user to assess the contribution of the original covariate rather than each dummy variable separately.

Partial covariate effects

In addition to assessing whether a covariate contributes to the fitted model, it is useful to describe the nature of that contribution. For this purpose, **statnnet** provides PCE plots.

Partial dependence plots describe how the average fitted response changes as one covariate is varied while the remaining covariates are held at their observed values. For covariate j , the partial dependence function is estimated as

$$\overline{\text{NN}}(x^{(j)}) = \frac{1}{n} \sum_{i=1}^n \text{NN}(x_i^{(j)}, \hat{\theta}),$$

where $x_i^{(j)}$ denotes the covariate vector for observation i with the j th covariate set to the value $x^{(j)}$.

The PCE is a finite-difference version of this quantity. For a d -unit increase in covariate j , the PCE is defined as

$$\hat{\beta}(x^{(j)}, d) = \overline{\text{NN}}(x^{(j)} + d) - \overline{\text{NN}}(x^{(j)}).$$

This quantity has an interpretation close to a regression coefficient: it estimates the average change in the fitted response associated with a d -unit increase in a covariate. The value of d can be chosen by the user; common choices are $d = 1$ or d equal to one empirical standard deviation of the covariate.

For binary covariates, the PCE corresponds to the average change in the fitted response when the covariate changes from 0 to 1. For continuous covariates, plotting $\hat{\beta}(x^{(j)}, d)$ over a sequence of values shows how the covariate effect varies across the covariate range. When the PCE is approximately constant, the fitted relationship is approximately linear in that covariate; when it varies substantially, the fitted model suggests a non-linear effect.

Uncertainty intervals for the PCE can be computed using the approximate sampling distribution of $\hat{\theta}$. For the delta method, the gradient of the PCE with respect to $\hat{\theta}$ is used:

$$\nabla_{\hat{\theta}} \hat{\beta}(x^{(j)}, d) = \frac{1}{n} \sum_{i=1}^n \left[\nabla_{\hat{\theta}} \text{NN}(x_i^{(j)} + d, \hat{\theta}) - \nabla_{\hat{\theta}} \text{NN}(x_i^{(j)}, \hat{\theta}) \right].$$

Hence,

$$\widehat{\text{Var}}(\hat{\beta}(x^{(j)}, d)) \approx \nabla_{\hat{\theta}} \hat{\beta}(x^{(j)}, d)^T \widehat{\text{Cov}}(\hat{\theta}) \nabla_{\hat{\theta}} \hat{\beta}(x^{(j)}, d).$$

This gives an analytic approximation to the variance, and, therefore, the standard error of the PCE at each plotted covariate value. Alternatively, uncertainty can be obtained by simulating parameter vectors from the approximate sampling distribution of $\hat{\theta}$ and recomputing the PCE for each simulated set of parameters as in [Mandel \(2013\)](#).

The package also provides a single point-estimate covariate effect by averaging the PCE over the observed values of the covariate:

$$\hat{\tau}_j = \frac{1}{n} \sum_{i=1}^n \hat{\beta}(x_{ij}, d).$$

This value is reported in the package summary output and is accompanied by a standard error. Together, the Wald-type summaries and PCE plots provide the two pieces of information usually sought from regression-style output: whether a covariate appears to contribute to the fitted model, and how the fitted response changes with that covariate.

3 Relationship to existing software

The role of **statnnet** is most clearly understood in relation to **nnet** itself. The `nnet()` function fits single-hidden-layer neural networks using the BFGS algorithm and provides methods for prediction, together with access to the fitted weights, residuals, objective value, convergence code, and Hessian. These quantities provide the computational basis for further statistical summaries, but **nnet** does not itself provide covariate-level Wald-type summaries or PCE plots with uncertainty estimates. The purpose of **statnnet** is to supply this additional layer for supported "nnet" objects.

This close dependence on **nnet** is a deliberate design choice. The package does not attempt to translate objects from several neural-network frameworks into a common representation, and it does not claim that the same covariance calculations are appropriate for arbitrary architectures or fitting objectives. Instead, the supported parameterisation, activation functions, weight ordering, objective, and convergence information are tied directly to those used by **nnet**. This narrower interface makes it possible to validate the required model information and to issue warnings when a fitted network lies outside the intended statistical setting.

As previously mentioned, there are also several R packages for model-agnostic interpretability, including packages that provide partial dependence plots, accumulated local effects, variable-importance measures, surrogate models, or Shapley-value-based explanations. Examples include packages such as **pdp** (?), **iml** (?), **DALEX** (?), and **vip** (?). These tools are valuable because they can be applied to many different fitted model classes. Their model-agnostic nature means that the same interpretability method can often be used for random forests, boosted trees, support vector machines, neural networks, and other predictive models. However, this generality also means that they are not designed to use the specific likelihood-based parameterisation of a shallow FNN. They are primarily designed to explain fitted prediction functions rather than to provide regression-style summaries based on the fitted neural-network parameters.

The approach in **statnnet** is different. It uses the fitted FNN parameterisation to compute Wald-type summaries, covariance-based standard errors, and PCE uncertainty intervals. The resulting output is therefore closer in spirit to the output from classical statistical models, such as `lm()` and `glm()`, than to purely model-agnostic explainability tools. This distinction is important. A model-agnostic partial dependence plot can describe how predictions change as a covariate varies, but it does not usually provide a covariate-level Wald-type summary based on the fitted neural-network parameters. Similarly, a variable-importance measure can rank predictors by predictive contribution, but it is not usually a model-based standard error for a covariate-effect estimate. **statnnet** provides these statistically motivated summaries for the specific case of supported single-hidden-layer models fitted using **nnet**.

Summary comparison

Table 1 summarises the relationship between **statnnet**, **nnet**, and selected model-agnostic interpretation packages. The comparison highlights that **statnnet** extends one specific fitting engine rather than providing another general-purpose fitting or explainability framework.

Table 1: Comparison of **statnnet** with **nnet** and selected model-interpretation packages.

Package	Fits neural networks	Model-agnostic	Parameter-based summaries	Wald-type tests
nnet	Yes, shallow FNNs	No	Limited	No
pdp	No	Yes	No	No
iml	No	Yes	No	No
DALEX	No	Yes	No	No
vip	No	Yes	No	No
statnnet	No; extends fitted nnet objects	No	Yes	Yes

4 Using statnnet

We now demonstrate the main workflow in **statnnet** using the insurance data considered in [McInerney and Burke \(2023\)](#). This section is intended to show the package as it would typically be used: a parsimonious, single-hidden-layer FNN is fitted using **nnet**, augmented as an object of class "statnnet", and then inspected using familiar statistical modelling functions. The aim is not to repeat the full applied analysis from [McInerney and Burke \(2023\)](#), but to show how the package moves from a fitted neural network to regression-style summaries, covariate-level tests, PCE plots, network visualisations, and predictions.

The main function is `statnnet()`, which takes a fitted "nnet" object and constructs the additional quantities required for statistical summaries:

```
statnnet(object, formula, data, ...)
```

The resulting object retains the fitted "nnet" model and stores the model matrix, response, likelihood information, covariance estimates, Wald-type summaries, covariate-effect estimates, and diagnostic quantities required by the package methods. Model fitting remains the responsibility of **nnet**; **statnnet** supplies a statistical-summary layer for the supported fitted object.

The current implementation is deliberately restricted to parsimonious single-hidden-layer FNNs fitted to tabular data, with continuous or binary response variables. The model is treated as a likelihood-based non-linear regression model, with a Gaussian likelihood for continuous responses and a Bernoulli likelihood for binary responses. Input variables may be continuous, binary, or categorical, with categorical variables represented through the corresponding dummy variables in the model matrix. The package does not support deep architectures, multiple hidden layers, or fitted objects produced by other neural-network packages.

Data

The data contain information on $n = 1338$ insurance beneficiaries. The response variable, `charges`, is the total medical expenses charged to an insurance plan. The explanatory variables are the beneficiary's age, sex, body mass index, number of children covered by the plan, smoking status, and region of residence. The version of the data included in **statnnet** is already centred, scaled, and dummy encoded, so the factor variables appear as binary input columns.

```
library(interpretnn)
library(nnet)

data("insurance", package = "interpretnn")

names(insurance)

#> [1] "age"           "sexmale"       "bmi"           "children"
#> [5] "smokeryes"    "regionnorthwest" "regionsoutheast" "regionsouthwest"
#> [9] "charges"

head(insurance[, c("age", "sexmale", "bmi", "children", "smokeryes", "charges")])

#>      age sexmale      bmi  children smokeryes  charges
#> 1 -1.4382265      0 -0.4531506 -0.90827406      1  0.2984722
#> 2 -1.5094011      1  0.5094306 -0.07873775      0 -0.9533327
```

```
#> 3 -0.7976553      1  0.3831636  1.58033487      0 -0.7284023
#> 4 -0.4417824      1 -1.3050431 -0.90827406      0  0.7195739
#> 5 -0.5129570      1 -0.2924471 -0.90827406      0 -0.7765118
#> 6 -0.5841316      0 -0.8073542 -0.90827406      0 -0.7856145
```

For this example, we model charges using all available covariates:

```
insurance_formula <- charges ~ .
```

The variables include both continuous covariates, such as age and bmi, and dummy variables representing categorical covariates, such as sexmale, smokeryes, and the regional indicators. The example is therefore useful for illustrating input-level summaries and the additional care required when a categorical covariate is represented by several dummy variables.

Fitting the neural network

We first fit a single-hidden-layer FNN using `nnet()`. Since the objective is non-convex, we use several random initialisations and retain the converged fit with the smallest fitted objective value:

```
set.seed(1237019)

nnet_fits <- replicate(
  10,
  nnet(
    insurance_formula,
    data = insurance,
    size = 2,
    linout = TRUE,
    decay = 0.01,
    maxit = 1000,
    Hess = TRUE,
    trace = FALSE
  ),
  simplify = FALSE
)

converged <- vapply(nnet_fits, function(fit) fit$convergence == 0, logical(1))
stopifnot(any(converged))

objective <- vapply(nnet_fits, function(fit) fit$value, numeric(1))
objective[!converged] <- Inf
nn <- nnet_fits[[which.min(objective)]]
```

The fitted model has two hidden nodes and uses the `nnet` weight-decay argument with decay = 0.01. The convergence check prevents a fit that reached the iteration limit from being selected solely because of its objective value. The statistical summaries below are conditional on the retained architecture, penalty, and local solution; they do not account for the choice among initialisations.

Creating a "statnnet" object

The object `nn` is a fitted "nnet" object. It can be augmented with the model information required by `statnnet`:

```
snn <- interpretnn(
  object = nn,
  formula = insurance_formula,
  data = insurance,
  response = "continuous"
)
```

The fitted "nnet" object does not always retain all of the information required to reconstruct the statistical model, so the original formula and training data are supplied explicitly. The constructor

Table 2: Main components of an object of class `statnnet`.

Component	Description
'object'	Original fitted object of class 'nnet'.
'weights'	Vector of fitted neural-network parameters.
'loss_val'	Value of the relevant loss function.
'n_inputs'	Number of input nodes.
'n_nodes'	Number of nodes in the single hidden layer.
'n_param'	Number of model parameters.
'n'	Number of observations in the training data.
'loglike'	Value of the model log-likelihood.
'BIC'	Bayesian information criterion for the fitted model.
'eff'	Point-estimate partial covariate effects and standard errors.
'wald'	Multiple-parameter Wald-type summaries for input nodes and covariates.
'wald_sp'	Single-parameter Wald-type summaries for individual weights.
'X'	Model matrix used by the fitted neural network.
'y'	Response vector.
'response'	Response type, such as continuous or binary.
'lambda'	Weight-decay value used in fitting.
'diagnostics'	Convergence and covariance-estimation diagnostics.

checks the fitted architecture, response type, convergence code, dimensions of the reconstructed model matrix, and availability of the quantities required for covariance estimation.

The most important components of a "statnnet" object are described in Table 2. The user will not usually need to access these components directly; they are stored so that the S3 methods can compute summaries, plots, and predictions from the same object.

Printing the fitted network

The `print()` method gives a compact overview of the fitted FNN:

```
print(snn)
```

The printed output reports the model call, response type, number of observations, number of input nodes, number of hidden nodes, and the value of the ridge penalty. This gives a quick check that the fitted network has the intended architecture before the statistical summaries are examined.

Regression-style summary

The `summary()` method gives the main regression-style summary produced by **statnnet**:

```
summary(snn)
```

The output contains one row for each input column in the fitted model matrix. For each input, **statnnet** reports a point-estimate PCE, its standard error, a Wald-type statistic, and a corresponding p-value. Thus, the summary combines two pieces of information that are commonly sought in regression modelling: an estimate of the size of the input effect and a test of whether that input appears to contribute to the fitted model.

For example, in the insurance data, variables such as age, bmi, children, and smokeryes are expected to contribute strongly to the fitted response. The summary output allows these effects to be inspected in a form that is closer to a classical regression summary than to the raw weight output usually obtained from neural-network software.

By default, `summary()` focuses on covariate-level summaries. Single-parameter summaries for individual neural-network weights can also be displayed:

```
summary(snn, wald_single_par = TRUE)
```

These individual-weight summaries are mainly diagnostic. For interpretation, the covariate-level summaries and the PCE plots are usually more informative than individual input-to-hidden-layer weights.

Analysis-of-variance-style summary

The `anova()` method provides covariate-level Wald-type summaries:

```
anova(snn)
```

For continuous and binary covariates, this corresponds to testing the group of weights connecting the covariate to the hidden layer. For categorical covariates represented by multiple dummy variables, such as region, the method tests all associated input nodes jointly. This gives a summary that is analogous in spirit to an analysis-of-variance table, where the contribution of the original covariate is assessed rather than the contribution of each dummy variable separately.

This is particularly useful when interpreting factor variables. For example, rather than separately inspecting the dummy variables corresponding to different regions, the `anova()` method gives a single covariate-level summary for region as a whole.

Network visualisation

The fitted architecture can be visualised using `plotnn()`:

```
plotnn(snn)
```

The plot displays the input nodes, hidden nodes, output node, and fitted connections. By default, statistically significant input nodes and weights are highlighted, while non-significant components are shown in a lighter colour. This gives a compact visual summary of the fitted network and the associated Wald-type summaries.

The significance threshold can be changed using the `alpha` argument:

```
plotnn(snn, alpha = 0.01)
```

Intercept terms can also be displayed if desired:

```
plotnn(snn, intercept = TRUE)
```

Partial covariate-effect plots

The `plot()` method produces PCE plots. For a continuous covariate such as `bmi`, the PCE plot shows how the estimated effect of a d -unit increase in BMI varies across the observed BMI range. The fitted object stores the model matrix in `snn$X`; in this example, `bmi` is the third input column:

```
plot(snn, which = 3, conf_int = FALSE)
```

If the PCE is approximately constant, the fitted model suggests an approximately linear effect for that covariate. If the PCE varies across the covariate range, the fitted model suggests a non-linear effect. This is one of the main advantages of the PCE plot: it gives an effect-size interpretation while still allowing the fitted neural network to represent non-linear relationships.

For binary covariates, the PCE corresponds to the average change in fitted response when the covariate changes from 0 to 1. For example, the effect of smoking status can be visualised using the fifth input column, `smokeryes`:

```
plot(snn, which = 5, conf_int = FALSE)
```

Several covariates can be plotted at once:

```
plot(
  snn,
  which = c(1, 3, 4, 5),
  conf_int = FALSE
)
```

The uncertainty intervals are computed using the approximate sampling distribution of the fitted parameters. The default uncertainty method can be changed using the `method` argument. For example, the delta-method intervals are obtained using:


```
plot(
  snn,
  which = 1,
  conf_int = TRUE,
  method = "deltamethod"
)
```

Alternatively, simulation from the approximate sampling distribution of the fitted parameters can be used:

```
plot(
  snn,
  which = 1,
  conf_int = TRUE,
  method = "mlesim"
)
```

Visualising interactions

Because FNNs can represent interactions between covariates, it can also be useful to examine whether a PCE changes across the values of another covariate. For example, the effect of `bmi` can be plotted separately for different smoking-status groups when the fitted object and plotting method support grouped PCE displays:

```
plot(
  snn,
  which = 3,
  by = 5,
  conf_int = TRUE
)
```

If the resulting PCE curves differ substantially between the two smoking-status groups, this suggests that the fitted network has captured an interaction between `bmi` and `smokeryes`. This type of plot is exploratory, but it is useful for communicating how the fitted neural network changes across different regions of the covariate space.

Prediction

The `predict()` method returns fitted values for the training data when no new data are supplied:

```
fitted_values <- predict(snn)

head(fitted_values)
```

Predictions for new observations can be obtained using the `newdata` argument:

```
new_insurance <- insurance[1:5, setdiff(names(insurance), "charges")]

predict(snn, newdata = new_insurance)
```

This preserves a prediction interface for `"statnnet"` objects that is consistent with the underlying `"nnet"` fit. The same object can therefore be used both for interpretation and prediction.

Summary of the workflow

This example illustrates the main role of **statnnet**: it augments a supported `"nnet"` fit with familiar statistical modelling tools. The fitted network can be printed, summarised, visualised, and used for prediction through standard R methods. The resulting output is not intended to replace predictive validation or the methodological checks discussed in [McInerney and Burke \(2023\)](#). Instead, it provides a practical interface for interpreting fitted shallow FNNs using regression-style summaries and covariate-effect visualisations.

5 Supported objects and methods

This section summarises the model types, object requirements, and user-facing methods supported by **statnnet**. Since the main workflow has already been demonstrated in Section 2.4, the emphasis here is on the deliberately narrow scope and interface rather than on another worked example.

Design considerations

The main design goal of **statnnet** is to make a supported "nnet" fit behave more like a familiar statistical model object in R. The package therefore uses an S3 interface, with methods such as `print()`, `summary()`, `anova()`, `vcov()`, `plot()`, and `predict()` defined for objects of class "statnnet". This keeps the user-facing workflow close to functions such as `lm()` and `glm()`, while allowing the fitted model itself to remain a neural network. The central function is `statnnet()`, which validates and augments a fitted "nnet" object with the model matrix, response, penalty value, likelihood quantities, covariance estimates, Wald-type summaries, PCE summaries, and diagnostics needed by the package methods. The original fit is retained so that prediction and fitted-model information remain tied to **nnet**.

Supported model types

The current version of **statnnet** is designed only for single-hidden-layer FNNs fitted to tabular data using `nnet()`. The package supports continuous and binary response variables. For continuous responses, the fitted network must use a linear output and is treated as a non-linear Gaussian regression model. For binary responses, it must use a single logistic output and is treated as a Bernoulli regression model.

The main supported architecture is the shallow FNN described in Section 2.2. This scope is deliberate: the inferential summaries implemented in the package are most naturally suited to parsimonious shallow networks, where the model can be interpreted as a flexible extension of classical regression. Skip-layer connections, multinomial softmax models, censored outputs, multiple output nodes, and models fitted by other neural-network packages are outside the initial scope. The package is not intended to provide general-purpose inference for deep or otherwise highly parameterised neural networks.

Input variables may be continuous, binary, or categorical. Categorical variables are represented internally through dummy variables in the model matrix. **statnnet** retains the mapping between dummy variables and the original covariates when the formula and data are supplied, allowing categorical predictors to be summarised at the covariate level rather than only at the dummy-variable level.

Requirements and diagnostic checks

The input to `statnnet()` must inherit from class "nnet" and must contain a supported architecture and response type. The formula and training data are required when they cannot be recovered unambiguously from the fitted object. These inputs are used to reconstruct the model matrix and response and to check that their dimensions agree with the fitted network.

Before computing covariance-based summaries, the package checks the **nnet** convergence code and the availability and numerical behaviour of the Hessian. The diagnostics record whether the optimiser reported convergence, whether the covariance calculation can be completed, and whether the relevant matrices are poorly conditioned or effectively singular. A failed convergence or covariance check prevents the corresponding Wald-type and uncertainty summaries from being reported; a warning is used for conditions where computation is possible but the output requires particular caution. These checks do not establish that a fitted model satisfies every regularity condition, but they prevent clearly unsuitable fits from being summarised without notice.

User-facing functions and methods

The main user-facing functions and methods are summarised in Table 3. Together, these functions add statistical summaries to a fitted "nnet" object while preserving the established fitting and prediction workflow.

The intended division of labour is explicit: **nnet** fits the model, `statnnet()` validates and augments it, and the remaining methods provide summaries, visualisations, and predictions from the

Table 3: Main user-facing functions and methods in `statnnet`.

Function	Purpose
<code>'statnnet()'</code>	Validates and augments a supported object of class <code>'nnet'</code> .
<code>'print()'</code>	Prints a compact description of the fitted network and its diagnostics.
<code>'summary()'</code>	Produces regression-style summaries, including PCE estimates and Wald-type summaries.
<code>'anova()'</code>	Produces covariate-level Wald-type summaries, including grouped summaries for categorical covariates.
<code>'vcov()'</code>	Returns the estimated covariance matrix when its diagnostic checks are satisfied.
<code>'plot()'</code>	Produces partial covariate-effect plots, with optional uncertainty intervals.
<code>'plotnn()'</code>	Produces a neural-network architecture diagram annotated with Wald-type summaries.
<code>'predict()'</code>	Returns fitted values or predictions for new observations.

augmented object. The methods are designed to be used in combination: `summary()` and `anova()` provide numerical summaries, `vcov()` exposes the covariance estimate, `plot()` and `plotnn()` provide graphical summaries, and `predict()` delegates prediction to the underlying `"nnet"` fit.

Output objects

The main object returned by `statnnet()` has class `"statnnet"`. This object contains the original `"nnet"` fit together with the data objects, likelihood quantities, covariance estimates, Wald-type summaries, covariate-effect summaries, and diagnostics required by the package methods. Users will typically interact with this object through the methods described above rather than by directly extracting components.

The summary and analysis-of-variance methods return structured output that can be printed, inspected, or stored for later use. Similarly, the plotting methods return graphical objects where appropriate, allowing users to further modify or combine the plots using standard R graphics workflows.

The overall aim is to provide an interface that feels familiar to users of standard statistical modelling functions while allowing the fitted model to remain a neural network. In this way, **statnnet** acts as a focused statistical companion to **nnet** rather than as a general interface to neural-network software.

6 Practical guidance and limitations

The summaries produced by **statnnet** are intended to help users interpret supported `"nnet"` fits in a regression-style workflow. However, they should be used with the same care that is required when interpreting any non-linear model, and with additional care due to the particular features of neural-network parameterisations. The main practical considerations are as follows.

First, **statnnet** is intended for shallow, parsimonious FNNs fitted to tabular data using **nnet**. The methodology implemented in the package is most appropriate when the fitted network is small enough to be inspected and when the number of hidden nodes is not excessive relative to the number of covariates and observations. In this setting, the model can be viewed as a flexible extension of classical regression. For large or highly overparameterised networks, the individual parameters are typically much harder to interpret, and covariance-based summaries may become unstable.

In practice, the number of hidden nodes should therefore be treated as part of the statistical modelling decision, not only as a predictive tuning parameter. Small values of q are easier to inspect and are more consistent with the regression-style interpretation targeted by the package. Increasing q may improve predictive flexibility, but it also increases the number of weights, can introduce redundant hidden nodes, and can make covariance-based summaries less stable. For this reason, sensitivity to the chosen hidden-layer size should be examined when the resulting summaries are substantively important.

Second, the use of a ridge penalty, or weight decay, is recommended. The penalty can improve numerical stability and reduce issues associated with near-singular Hessian matrices. In **statnnet**, the weight-decay value recorded by the fitted `"nnet"` object is also incorporated into the computation of the covariance matrix and related Wald-type summaries. The value of the penalty should therefore be treated as part of the fitted model. The package reports the value of λ used in fitting so that the interpretation of the resulting summaries is tied to the fitted regularised network. Different values of λ can lead to different fitted weights and different covariance estimates, so the penalty should be reported and, where appropriate, examined as part of a sensitivity analysis.

Third, the fitted neural network may depend on the optimisation procedure. Neural-network objective functions are generally non-convex, and different random initialisations can lead to different

fitted parameter values. For this reason, users should use multiple random initialisations when fitting the model, inspect whether the fitted loss is stable, and avoid placing too much emphasis on summaries from a single unstable fit. The applied workflow therefore fits the model from several initial values and checks the convergence code before selecting a solution. This reduces the chance that the reported summaries are based on a poor local optimum, but it does not account for the additional uncertainty introduced by selecting among the fitted solutions.

The reported Wald-type quantities are also conditional on the chosen hidden-layer size and penalty. If these quantities are selected using the same data, their uncertainty is not incorporated into the reference distributions reported by **statnnet**. The summaries should therefore not be interpreted as post-selection inference. Where architecture or penalty choices are data driven, their stability should be examined separately and the selection procedure should be reported.

The covariance calculation depends on the local curvature of the fitted objective. Redundant hidden nodes or weakly identified parameters can produce a near-singular Hessian even when `nnet()` reports optimiser convergence. For this reason, the convergence and covariance diagnostics should be examined before interpreting any standard error, Wald-type statistic, or uncertainty interval. When the relevant matrix cannot be inverted reliably, **statnnet** does not report the affected covariance-based summaries.

Fourth, individual neural-network weights should be interpreted cautiously. A single input-to-hidden-layer weight is only one component of the fitted function, and its meaning depends on the remaining weights in the network. In addition, hidden nodes can be permuted without changing the fitted input-output function, and sign or label symmetries can lead to equivalent parameterisations. For this reason, **statnnet** reports individual-weight summaries mainly as diagnostic quantities. In most applications, input-level summaries, covariate-level summaries, and PCE plots will be more useful for interpretation.

Fifth, categorical covariates should generally be interpreted at the covariate level rather than only at the dummy-variable level. When a factor is represented by several dummy variables, separate input-level summaries can be difficult to interpret in isolation. The `anova()` method therefore groups the associated input nodes and provides a covariate-level Wald-type summary. This gives an output closer in spirit to an analysis-of-variance table for a standard regression model.

Sixth, PCE plots should be interpreted as summaries of the fitted model, not as causal effects. A PCE describes how the fitted response changes when a covariate is varied while the remaining covariates are averaged over their observed values. As with partial dependence plots more generally, interpretation can be more difficult when covariates are strongly correlated or when the plot evaluates combinations of covariate values that are rare in the observed data. The plots are therefore best viewed as descriptive tools for understanding the fitted prediction function.

Finally, the inferential summaries produced by **statnnet** do not replace model checking or predictive validation. Users should still assess predictive performance using appropriate validation procedures, examine the sensitivity of the fitted model to architecture and penalty choices, and consider whether the chosen neural network is appropriate for the scientific or applied question. The package makes fitted shallow FNNs more interpretable, but the quality of the resulting interpretation depends on the quality and stability of the fitted model.

7 Conclusion

The **statnnet** package provides a statistical-summary layer for supported single-hidden-layer neural networks fitted using **nnet**. Rather than introducing a new optimiser or a common interface for several neural-network frameworks, the package augments a fitted "nnet" object with the information required for regression-style summaries, diagnostic checks, and covariate-effect visualisations. In this way, **statnnet** makes a deliberately restricted class of shallow neural networks behave more like standard statistical model objects in R.

The package implements regression-style summaries, Wald-type summaries for individual weights and groups of weights, analysis-of-variance-style summaries for covariates, PCE plots with uncertainty estimates, architecture diagrams annotated with statistical summaries, and prediction methods for new observations. These tools complement **nnet** by providing outputs that are more familiar to users of classical regression models.

The statistical methodology underlying these summaries is developed in [McInerney and Burke \(2023\)](#). The contribution of the present article is to describe the corresponding R implementation and demonstrate how the methodology can be used in practice through a coherent package workflow. By combining the flexibility of a parsimonious single-hidden-layer network with regression-style output, **statnnet** provides a practical interface for statisticians who wish to use supported "nnet" models as statistical modelling tools rather than purely predictive algorithms.

Future development will remain centred on the `nnet` setting, with possible extensions including alternative uncertainty estimators and additional diagnostics for assessing the stability of fitted models. Support for deep-learning frameworks or arbitrary neural-network architectures is not an objective of the package. More generally, `statnnet` provides a focused extension for integrating small "nnet" models more naturally into the statistical modelling workflow in R.

8 Acknowledgements

This publication has emanated from research conducted with the financial support of Science Foundation Ireland under Grant number 18/CRT/6049. The second author was supported by the Confirm Smart Manufacturing Centre (<https://confirm.ie/>) funded by Science Foundation Ireland (Grant Number: 16/RC/3918). For the purpose of Open Access, the authors have applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

Bibliography

- Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539. [p1]
- M. Mandel. Simulation-based confidence intervals for functions with complicated derivatives. *The American Statistician*, 67(2):76–81, 2013. doi: 10.1080/00031305.2013.783880. [p4]
- A. McInerney and K. Burke. Feedforward neural networks as statistical models: Improving interpretability through uncertainty quantification. 2023. [p1, 2, 3, 5, 9, 12]
- M. T. Ribeiro, S. Singh, and C. Guestrin. Why should i trust you?: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16*, page 1135–1144, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342322. doi: 10.1145/2939672.2939778. URL <https://doi.org/10.1145/2939672.2939778>. [p1]
- E. Strumbelj and I. Kononenko. An efficient explanation of individual classifications using game theory. *The Journal of Machine Learning Research*, 11:1–18, 2010. [p1]
- W. N. Venables and B. D. Ripley. *Modern Applied Statistics with S*. Springer, New York, fourth edition, 2002. ISBN 0-387-95457-0. [p1]

Andrew McInerney
University of Limerick
Department of Mathematics and Statistics
Limerick, Ireland
ORCID: 0000-0002-2348-2159
andrew.mcinerney@ul.ie

Kevin Burke
University of Limerick
Department of Mathematics and Statistics
Limerick, Ireland
kevin.burke@ul.ie